Labo 3 – Dataverwerking I

Van bestand tot nette tabel

Cursustekst bij het notebook `lab3_data_processing_I.py`

dr. Maarten Slembrouck

Waarover gaat dit labo?

In de vorige twee labo's leerde je de basis van Python: variabelen, functies, lijsten, dictionaries en lussen. Op het einde van labo 2 las je `athletes.csv` in met de module `csv` en telde je met een `for`-lus hoeveel atleten er per positie zijn. Dat werkte, maar het kostte veel regels code. Vanaf dit labo gebruik je **pandas**, een bibliotheek die precies voor dit soort werk gemaakt is.

Het verhaal blijft hetzelfde: je bent de nieuwe data-analist van een jeugdclub voor voetbal en atletiek. De hoofdtrainer geeft je drie exportbestanden. Die zijn bewust rommelig, zoals echte exports:

- `athletes.csv`: één rij per atleet (ID, naam, geboortedatum, geslacht, positie, lengte, gewicht);
- `tests.csv`: resultaten van fysieke testen (sprint over 10 m en 30 m, spronghoogte, Yo-Yo-niveau) op drie testmomenten;
- `hr_session.csv`: hartslag en snelheid per seconde tijdens één training (die gebruik je pas in labo 4).

Na dit labo kun je een dataset inladen en verkennen, rijen en kolommen selecteren en filteren, nieuwe kolommen berekenen en de data opkuisen tot je ze kunt vertrouwen.

Hoe gebruik je deze tekst? Lees ze naast het notebook. Elk codevoorbeeld hieronder is echt uitgevoerd op de data van de club; het kader *uitvoer* toont wat Python teruggaf. Typ de voorbeelden zelf over in een lege cel: je leert meer van code die je zelf typt dan van code die je leest. De oefeningen zelf staan in het notebook; deze tekst legt uit wat je nodig hebt om ze op te lossen.

1 Een paar Python-begrippen die je vandaag nodig hebt

Pandas is gewone Python. Vier dingen uit de vorige labo's komen voortdurend terug:

- **Een functie aanroepen** doe je met haakjes: `round(5.3456, 2)`. De waarden tussen de haakjes heten *argumenten*.
- **Argumenten met een naam.** Sommige argumenten geef je door met hun naam: `round(number=5.3456, ndigits=2)`. In pandas zie je dat heel vaak, bv. `ascending=False`.
- **Een methode** is een functie die bij een object hoort. Je roept ze aan met een punt: `"Forward".lower()` geeft `"forward"`. Een tabel in pandas heeft tientallen methodes.
- **Lijsten en dictionaries**: `["name", "position"]` is een lijst, `{"fwd": "Forward"}` is een dictionary die een sleutel aan een waarde koppelt.

2 Wat is pandas?

Pandas is een **bibliotheek**: een verzameling code die iemand anders geschreven heeft en die je in je eigen programma kunt gebruiken. Je laadt ze met `import`. Iedereen gebruikt de afkorting `pd`:

```
import pandas as pd
```

Pandas draait rond twee soorten objecten:

- een **DataFrame** is een tabel met rijen en kolommen, zoals een werkblad in Excel;
- een **Series** is één kolom van zo'n tabel.

Elke rij heeft een label, de **index**. Standaard is dat gewoon een nummer, beginnend bij 0. Je kunt zelf een kleine DataFrame maken uit een dictionary: elke sleutel wordt een kolomnaam, elke lijst de waarden van die kolom.

```
mini = pd.DataFrame({
    "name": ["Yana", "Stan", "Emma"],
    "position": ["Defender", "Midfielder", "Forward"],
    "height_cm": [167.1, 158.1, 171.0],
})
mini
```

uitvoer

```
   name    position  height_cm
0  Yana    Defender      167.1
1  Stan  Midfielder      158.1
2  Emma     Forward      171.0
```

De kolom links (0, 1, 2) is de index; die is geen echte kolom met data. Eén kolom opvragen geeft een Series:

```
mini["height_cm"]
```

uitvoer

```
0    167.1
1    158.1
2    171.0
Name: height_cm, dtype: float64
```

Kernidee

Een DataFrame is een tabel, een Series is één kolom. Bijna alles wat je vandaag doet, is: een DataFrame nemen, er iets mee doen, en een nieuwe DataFrame of Series terugkrijgen.

Het grote voordeel tegenover lijsten: een berekening op een Series werkt meteen op *elke* rij, zonder `for`-lus.

```
mini["height_cm"] / 100
```

uitvoer

```
0    1.671
1    1.581
2    1.710
Name: height_cm, dtype: float64
```

3 Een CSV-bestand inladen

Een CSV-bestand (*comma-separated values*) is gewone tekst: één regel per rij, met komma's tussen de waarden. De eerste regel bevat de kolomnamen. Zo ziet het begin van `tests.csv` eruit als tekst:

uitvoer
```
test_id,athlete_id,test_moment,date,sprint_10m_s,sprint_30m_s,cmj_height_cm,yoyo_level
1,A001,1,2026-09-14,2.03,5.34,20.6,19.0
2,A001,2,2026-10-19,2040.0,5.32,21.2,20.0
3,A002,1,2026-09-14,2360.0,5.97,30.4,14.0
```

Voor Python is dat gewoon tekst: het weet niet dat `5.34` een getal is, of dat `date` een datum bevat. `pd.read_csv` leest het bestand en maakt er een DataFrame van, met een gepast datatype per kolom:

```
athletes_raw = pd.read_csv(f"{DATA_DIR}/athletes.csv")
athletes_raw.head()
```

uitvoer

	athlete_id	name	birthdate	gender	position	height_cm	weight_kg
0	A001	Yana De Backer	2010-05-27	F	defender	167.1	65,5
1	A002	Stan Mertens	2012-02-19	M	midfielder	158.1	59,6
2	A003	Amber Jacobs	2010-08-19	F	Defender	168.0	63,9
3	A004	Bram Wouters	2009-04-07	M	MID	177.8	68,7
4	A005	Emma Verhoeven	2010-10-18	F	goalkeeper	170.1	61,1

`DATA_DIR` is een variabele die bovenaan het notebook gemaakt wordt: de map met de data. Lokaal is dat de map `data/`, online een adres op GitHub. Met `f"{DATA_DIR}/athletes.csv"` (een *f-string*) plak je de bestandsnaam erachter.

Opgelet: namen in marimo

In marimo mogen twee cellen niet dezelfde variabele maken. Geef daarom elke stap een eigen naam: `athletes_raw` voor de ruwe data, `athletes_clean` voor de opgekuiste versie. Dat maakt je code bovendien leesbaarder.

4 Een eerste indruk van de data

Voor je iets met een dataset doet, bekijk je ze. Vijf hulpmiddelen geven snel een overzicht.

`.head()` toont de eerste vijf rijen (of `.head(10)` voor tien). Dat zag je hierboven al.

`.shape` geeft het aantal rijen en kolommen. Let op: er staan geen haakjes achter, want `shape` is een eigenschap en geen methode.

```
athletes_raw.shape
```

uitvoer
```
(43, 7)
```

Het resultaat is een *tuple*: een vaste reeks waarden tussen ronde haakjes. `athletes_raw.shape[0]` is het aantal rijen (43), `athletes_raw.shape[1]` het aantal kolommen (7).

`.dtypes` toont welk datatype pandas per kolom gekozen heeft:

```
athletes_raw.dtypes
```

uitvoer
```
athlete_id        str
name              str
```

uitvoer

```
birthdate         str
gender            str
position          str
height_cm     float64
weight_kg         str
dtype: object
```

`int64` is een geheel getal, `float64` een kommagetal en `str` tekst. Valt je iets op? `weight_kg` is tekst, terwijl gewicht een getal zou moeten zijn. Dat is het eerste probleem dat je vandaag oplost. Ook `birthdate` is tekst; pandas herkent een datum niet vanzelf.

`.info()` combineert dat met het aantal ingevulde (*non-null*) waarden per kolom. `.describe()` geeft statistieken voor de getallenkolommen: aantal, gemiddelde (`mean`), standaardafwijking (`std`), minimum, kwartielen en maximum.

```
athletes_raw.describe()
```

uitvoer

```
        height_cm
count   41.000000
mean   163.712195
std      9.465891
min    141.100000
25%    157.100000
50%    163.900000
75%    170.100000
max    179.200000
```

`count` is 41 voor `height_cm`, terwijl er 43 atleten zijn: bij twee atleten ontbreekt de lengte. Zo'n tabel is een snelle manier om ontbrekende waarden en onmogelijke waarden op te sporen.

`.value_counts()` telt hoe vaak elke waarde in een kolom voorkomt. Wat je in labo 2 met een lus en een dictionary deed, is nu één regel:

```
athletes_raw["position"].value_counts()
```

uitvoer

```
position
FWD           6
defender      5
forward       5
midfielder    4
MID           4
goalkeeper    4
Midfielder    4
Defender      3
defender      3
Forward       2
GK            1
forward       1
DEF           1
Name: count, dtype: int64
```

De club heeft maar vier posities, maar er staan 13 verschillende schrijfwijzen in de kolom. Dat is het tweede probleem van vandaag.

5 Kolommen selecteren

Eén kolom selecteer je met vierkante haakjes en de kolomnaam. Je krijgt een Series terug. Meerdere kolommen selecteer je met een *lijst* van kolomnamen tussen de haakjes; je krijgt dan een DataFrame terug.

```
athletes_raw[["name", "position"]].head(3)
```

uitvoer

```
             name    position
0  Yana De Backer   defender
1    Stan Mertens  midfielder
2    Amber Jacobs    Defender
```

Opgelet: dubbele haakjes

De buitenste haakjes betekenen "selecteer", de binnenste maken een lijst. `athletes_raw["name", "position"]` (met enkele haakjes) geeft een `KeyError`: pandas zoekt dan één kolom met de vreemde naam `("name", "position")`.

6 Rijen filteren

Filteren gebeurt in twee stappen. Eerst schrijf je een **voorwaarde**. Een vergelijking op een Series geeft een nieuwe Series met `True` of `False` voor elke rij:

```
is_tall = athletes_raw["height_cm"] > 175
is_tall.head()
```

uitvoer

```
0    False
1    False
2    False
3     True
4    False
Name: height_cm, dtype: bool
```

Zo'n Series van `True`/`False` heet een **masker**. Daarna geef je dat masker aan `.loc[...]`. Je houdt alleen de rijen waar het masker `True` is:

```
athletes_raw.loc[is_tall, ["name", "position", "height_cm"]]
```

uitvoer

```
                name    position  height_cm
3       Bram Wouters         MID      177.8
6     Lotte Janssens          GK      176.3
9   Mathis Verhoeven   defender      179.2
17     Julie De Smet  goalkeeper      177.0
25   Amber Verhoeven     forward      178.8
```

`.loc[rijen, kolommen]` selecteert dus rijen *en* kolommen in één keer. Laat je het tweede deel weg, dan krijg je alle kolommen.

6.1 Filteren op tekst

Met `==` vergelijk je tekst. De vergelijking is exact: hoofdletters en spaties tellen mee.

```
only_forwards_raw = athletes_raw.loc[athletes_raw["position"] == "Forward"]
only_forwards_raw.shape
```

uitvoer

```
(2, 7)
```

Maar 2 rijen, terwijl `value_counts()` ook `"forward"`, `"forward "` en `"FWD"` toonde. Filteren op tekst werkt pas goed als de labels opgekuist zijn (sectie 8).

6.2 Filteren op datum

`birthdate` is tekst. Met `pd.to_datetime` zet je het om naar echte datums, die je kunt vergelijken. Wie na 1 september 2010 geboren is, is jonger dan 16 in het seizoen 2026:

```
younger_than_16 = pd.to_datetime(athletes_raw["birthdate"]) > "2010-09-01"
younger_than_16.sum()
```

uitvoer

```
28
```

`.sum()` op een masker telt het aantal `True`-waarden, want Python rekent `True` als 1 en `False` als 0. Er zijn dus 28 atleten jonger dan 16.

6.3 Voorwaarden combineren

Twee voorwaarden combineer je met `&` (en), `|` (of) en `~` (niet). Zet elke voorwaarde tussen haakjes:

```
tall_girls = athletes_raw.loc[
    (athletes_raw["gender"] == "F") & (athletes_raw["height_cm"] > 170)
]
tall_girls[["name", "gender", "height_cm"]]
```

uitvoer

```
                name gender  height_cm
4    Emma Verhoeven      F      170.1
6    Lotte Janssens      F      176.3
17    Julie De Smet      F      177.0
21    Hanne Willems      F      172.8
24     Roos Lemmens      F      171.3
25  Amber Verhoeven      F      178.8
39   Hanne Michiels      F      173.3
```

Opgelet: `and` werkt hier niet

In gewone Python schrijf je `and` en `or`. Op een Series werkt dat niet: je krijgt een `ValueError` (*The truth value of a Series is ambiguous*). Gebruik `&` en `|`, en vergeet de haakjes rond elke voorwaarde niet. Zonder haakjes rekent Python eerst `&` uit, en dan klopt de voorwaarde niet meer.

6.4 Sorteren

`.sort_values("kolom")` sorteert de rijen van klein naar groot. Met `ascending=False` keer je de volgorde om. De index sorteert mee: zo zie je nog uit welke rij elke waarde kwam.

```
tests_raw = pd.read_csv(f"{DATA_DIR}/tests.csv")
slowest_10m = tests_raw.sort_values("sprint_10m_s", ascending=False)
slowest_10m[["athlete_id", "sprint_10m_s"]].head()
```

uitvoer

```
    athlete_id  sprint_10m_s
2         A002        2360.0
67        A027        2220.0
106       A041        2160.0
83        A033        2120.0
7         A004        2100.0
```

Een sprint van 2360 seconden over 10 m? Die waarden staan in *milliseconden* in plaats van seconden (2,36 s werd 2360). Dat is het derde probleem: dezelfde fout zit ook in `sprint_30m_s`.

Probeer zelf

Hoeveel atleten zijn langer dan 180 cm? Gebruik een masker en `.sum()`. Sorteer daarna `athletes_raw` van lang naar kort en bekijk de eerste vijf rijen.

7 Nieuwe kolommen berekenen

Een nieuwe kolom maken is een toewijzing: `df["nieuwe_kolom"] = ...`. Rechts van het gelijkheidsteken staat meestal een berekening op andere kolommen. Die berekening gebeurt voor alle rijen tegelijk.

```
speed = tests_raw[["athlete_id", "sprint_30m_s"]].copy()
speed["speed_kmh"] = 30 / speed["sprint_30m_s"] * 3.6
speed.head(3)
```

uitvoer

```
  athlete_id  sprint_30m_s  speed_kmh
0       A001          5.34  20.224719
1       A001          5.32  20.300752
2       A002          5.97  18.090452
```

30 meter gedeeld door de tijd geeft meter per seconde; maal 3,6 geeft kilometer per uur. Merk op dat `30 / speed["sprint_30m_s"]` een getal deelt door een hele kolom: pandas doet de deling voor elke rij.

Waarom `.copy()`?

`speed = tests_raw[...]` maakt soms geen nieuwe tabel, maar een *venster* op de oude. Als je daar een kolom aan toevoegt, weet je niet zeker of de oorspronkelijke tabel mee verandert. Met `.copy()` maak je een echte, aparte kopie. In het notebook begin je daarom oefeningen met `athletes_raw.copy()`: zo blijft de ruwe data onaangeroerd.

7.1 Tekst omzetten naar een getal

`weight_kg` bevat tekst zoals `"57,5"`, met een komma als decimaalteken. Dat gebeurt vaak bij exports uit Europese software. Python verwacht een punt. Je lost dat op in twee stappen:

```
weight_kg_numeric = athletes_raw["weight_kg"].str.replace(",", ".").astype(float)
weight_kg_numeric.head(3)
```

uitvoer

```
0    65.5
1    59.6
2    63.9
Name: weight_kg, dtype: float64
```

- `.str` geeft toegang tot tekstmethodes voor *elke* rij; `.str.replace(",", ".")` vervangt de

komma door een punt.

- `.astype(float)` zet de tekst om naar een kommagetal. Probeer je dat zonder eerst de komma te vervangen, dan krijg je een `ValueError`: `"57,5"` is voor Python geen getal.

Je kunt methodes zo achter elkaar zetten (*chaining*): elke methode werkt op het resultaat van de vorige.

7.2 Rekenen met datums

Voor de leeftijd trek je de geboortedatum af van een vaste referentiedatum. Een vaste datum (en niet "vandaag") zorgt ervoor dat je resultaten morgen nog dezelfde zijn.

```
REFERENCE_DATE = pd.Timestamp("2026-09-14")
birthdates = pd.to_datetime(athletes_raw["birthdate"])
(REFERENCE_DATE - birthdates).head(3)
```

uitvoer

```
0   5954 days
1   5321 days
2   5870 days
Name: birthdate, dtype: timedelta64[us]
```

Het verschil tussen twee datums is een *tijdsduur* (een `Timedelta`). Met `.dt.days` haal je het aantal dagen eruit, en met `// 365` deel je door 365 en hou je alleen het gehele deel:

```
age_years = (REFERENCE_DATE - birthdates).dt.days // 365
age_years.head(3)
```

uitvoer

```
0    16
1    14
2    16
Name: birthdate, dtype: int64
```

`//` is de *gehele deling*: `5954 // 365` is 16, niet 16,31. Zo krijg je een leeftijd in hele jaren, zoals je die gewoon bent.

8 De data opkuisen

Opkuisen betekent: de problemen die je bij het verkennen vond één voor één oplossen, en voor elke keuze kunnen uitleggen waarom.

8.1 Ontbrekende waarden

Een lege cel in een CSV-bestand wordt in pandas `NaN` (*not a number*). Met `.isna()` krijg je een masker dat `True` is waar een waarde ontbreekt; `.isna().sum()` telt ze per kolom:

```
tests_raw.isna().sum()
```

uitvoer

```
test_id          0
athlete_id       0
test_moment      0
date             0
sprint_10m_s     0
sprint_30m_s     6
cmj_height_cm    6
```

```
yoyo_level       6
dtype: int64
```
uitvoer

Je hebt twee keuzes:

- `.dropna()` gooit de rijen met een ontbrekende waarde weg. Redelijk als het om weinig rijen gaat en je de waarde niet goed kunt schatten (bv. een ontbrekend geslacht).
- `.fillna(waarde)` vult de lege plaatsen in, bv. met het gemiddelde van de kolom. Redelijk als je een goede schatting hebt en de rij nodig hebt voor je analyse.

Kernidee

Er is geen regel die altijd klopt. Beslis per kolom en schrijf op *waarom*. "Alles met 0 opvullen" is bijna altijd fout: een ontbrekende spronghoogte is niet hetzelfde als een sprong van 0 cm.

8.2 Tekstlabels opkuisen

De posities staan op 13 manieren in de data. Twee tekstmethodes lossen het grootste deel op: `.str.strip()` haalt spaties aan het begin en einde weg, `.str.lower()` zet alles in kleine letters.

```python
position_normalized = athletes_raw["position"].str.strip().str.lower()
position_normalized.value_counts()
```

```
position
defender      11
midfielder     8
forward        8
fwd            6
mid            4
goalkeeper     4
gk             1
def            1
Name: count, dtype: int64
```
uitvoer

Van 13 naar 8 varianten. Wat overblijft zijn afkortingen. Die vertaal je met een dictionary en `.map(...)`: elke waarde wordt opgezocht als sleutel en vervangen door de bijhorende waarde.

```python
POSITION_MAP = {
    "forward": "Forward", "fwd": "Forward",
    "midfielder": "Midfielder", "mid": "Midfielder",
    "defender": "Defender", "def": "Defender",
    "goalkeeper": "Goalkeeper", "gk": "Goalkeeper",
}
position_clean = position_normalized.map(POSITION_MAP)
position_clean.value_counts()
```

```
position
Forward       14
Defender      12
Midfielder    12
Goalkeeper     5
Name: count, dtype: int64
```
uitvoer

Opgelet

Een waarde die niet in de dictionary staat, wordt door `.map` vervangen door `NaN`. Controleer daarom altijd met `.value_counts()` of `.isna().sum()` dat er niets verloren ging.

Om de opgekuiste kolom in de tabel te zetten, overschrijf je de oude kolom met een toewijzing, op een kopie van de ruwe tabel.

8.3 Getallen in de verkeerde eenheid

Geen mens loopt 10 of 30 meter in minder dan 1 seconde, en geen enkele sprint duurt 20 seconden. Een waarde boven 20 staat dus in milliseconden en moet gedeeld worden door 1000. Dat doe je met `.where(voorwaarde, anders)`: waar de voorwaarde `True` is, blijft de waarde staan; waar ze `False` is, komt de waarde van `anders` in de plaats.

```
def fix_sprint_units(seconds_column):
    return seconds_column.where(seconds_column < 20, seconds_column / 1000)

sprint_10m_fixed = fix_sprint_units(tests_raw["sprint_10m_s"])
sprint_10m_fixed.describe()
```

uitvoer

```
count    114.000000
mean       2.099912
std        0.149415
min        1.730000
25%        1.975000
50%        2.090000
75%        2.200000
max        2.470000
Name: sprint_10m_s, dtype: float64
```

Het maximum is nu 2,47 seconden: realistisch. Omdat je dezelfde stap op twee kolommen nodig hebt, staat ze in een **functie**. Zo schrijf je de regel één keer en gebruik je ze zo vaak als nodig.

8.4 Dubbels

`.duplicated(subset=[...])` geeft `True` voor elke rij die dezelfde waarden heeft in de opgegeven kolommen als een rij *erboven*. `.drop_duplicates(subset=[...], keep="first")` gooit die rijen weg en houdt het eerste exemplaar. Een klein voorbeeld:

```
sessions = pd.DataFrame({
    "name": ["Yana", "Stan", "Yana"],
    "day": ["ma", "ma", "ma"],
    "minutes": [60, 45, 60],
})
sessions.duplicated(subset=["name", "day"])
```

uitvoer

```
0    False
1    False
2     True
dtype: bool
```

In `athletes.csv` staat één atleet twee keer, met dezelfde naam en geboortedatum maar twee verschillende ID's. Een fout bij het invoeren. Daarom zoek je dubbels op *naam en geboortedatum*, niet op ID.

8.5 Alles samen: één pipeline

Op het einde van het labo zet het notebook alle stappen achter elkaar in één blok code. Zo'n reeks stappen heet een **pipeline**. Twee nieuwe dingen kom je daar tegen:

- `.assign(kolom=...)` maakt (of overschrijft) een kolom en geeft een nieuwe DataFrame terug. Je kunt er dus een volgende methode achter zetten, wat met `df["kolom"] = ...` niet kan.
- `lambda d: ...` is een korte functie zonder naam. In `.assign(bmi=lambda d: d["weight_kg"] / ...)` staat `d` voor de tabel *zoals ze op dat moment in de pipeline is*. Zo kun je een kolom gebruiken die een stap eerder gemaakt is.

```
athletes_clean = (
    athletes_raw.assign(
        position=position_clean,
        weight_kg=weight_kg_numeric,
        age=age_years,
    )
    .assign(bmi=lambda d: d["weight_kg"] / (d["height_cm"] / 100) ** 2)
    .drop_duplicates(subset=["name", "birthdate"], keep="first")
    .reset_index(drop=True)
)
```

```
athletes_clean[["name", "position", "weight_kg", "age", "bmi"]].head(3)
```

uitvoer

	name	position	weight_kg	age	bmi
0	Yana De Backer	Defender	65.5	16	23.457861
1	Stan Mertens	Midfielder	59.6	14	23.844187
2	Amber Jacobs	Defender	63.9	16	22.640306

De haakjes rond de hele pipeline laten toe om elke stap op een nieuwe regel te zetten. `.reset_index(drop=True)` nummert de rijen opnieuw van 0 tot het einde, want na het verwijderen van een dubbel ontbreekt er een nummer.

9 Vooruitblik: de uitdaging

De uitdaging op het einde van het notebook vraagt de 5 snelste atleten per positie. Daarvoor moet je twee tabellen samenvoegen (`athletes_clean` heeft de positie, `tests_clean` de sprinttijden) en per groep rangschikken. De hints in het notebook tonen de drie functies die je daarvoor nodig hebt: `groupby`, `merge` en `.head(5)` per groep. Die worden in labo 4 grondig uitgelegd; probeer de uitdaging nu al met de hints, en kijk volgende week of je ze beter begrijpt.

10 Samenvatting

Code	Wat het doet
`pd.read_csv(pad)`	CSV-bestand inlezen als DataFrame
`df.head()`, `df.shape`, `df.dtypes`	eerste rijen, (rijen, kolommen), datatypes
`df.info()`, `df.describe()`	overzicht, statistieken van getallenkolommen
`df["kol"].value_counts()`	hoe vaak elke waarde voorkomt
`df["kol"]`, `df[["a", "b"]]`	één kolom (Series), meerdere kolommen (DataFrame)
`df.loc[masker, kolommen]`	rijen filteren en kolommen kiezen
`(voorw1) & (voorw2)`, `|`, `~`	voorwaarden combineren: en, of, niet
`df.sort_values("kol", ascending=False)`	sorteren, van groot naar klein
`df["nieuw"] = ...`, `df.assign(...)`	nieuwe kolom maken
`.str.replace()`, `.str.strip()`, `.str.lower()`	tekstbewerkingen op elke rij
`.astype(float)`	datatype omzetten
`pd.to_datetime()`, `.dt.days`	tekst naar datum, aantal dagen uit een tijdsduur
`.isna().sum()`, `.dropna()`, `.fillna()`	ontbrekende waarden tellen, weggooien, opvullen
`.map(dictionary)`	waarden vertalen met een dictionary
`.where(voorwaarde, anders)`	waarden vervangen waar de voorwaarde `False` is
`.duplicated()`, `.drop_duplicates()`	dubbels vinden, verwijderen

Veelgemaakte fouten

- `KeyError`: een kolomnaam bestaat niet. Controleer de spelling met `df.columns`.
- `and`/`or` in plaats van `&`/`|`, of geen haakjes rond elke voorwaarde.
- Een kolom wijzigen zonder `.copy()` van de tabel te maken.
- Vergeten dat `==` op tekst exact vergelijkt (spaties, hoofdletters).
- In marimo dezelfde variabelenaam in twee cellen gebruiken.

Begrippen

DataFrame tabel met rijen en kolommen.

Series één kolom van een DataFrame.

index de labels van de rijen (standaard 0, 1, 2, . . .).

datatype soort waarde in een kolom: `int64`, `float64`, `str`, datum.

masker Series met `True`/`False`, gebruikt om rijen te filteren.

NaN ontbrekende waarde.

pipeline reeks bewerkingen die na elkaar op de data worden uitgevoerd.